HW6 – Design D1

Team Members:

Noah Schumacher

Ryan Broyles

Manjinder Kaur

Minh Nguyen

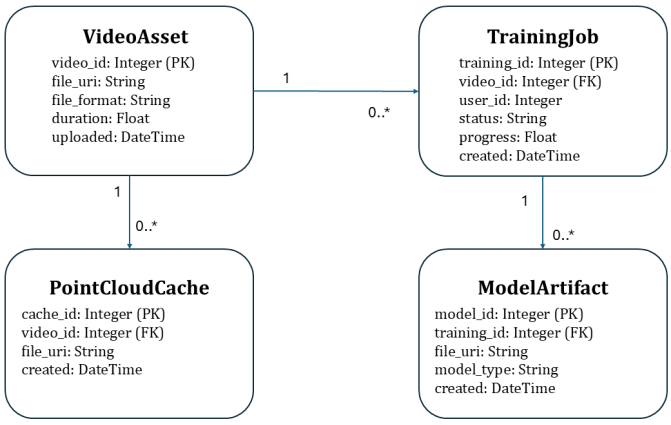
Akshat Chaturvedi

SkillForge

Goal: A web interface for developing, configuring, and tracking training robot skills from videos of human demonstrations

Components Expanded: D1 expands on the component design of the E2, E3, and E4 components from the high-level D0 design. Components E1, C1, C2, C3, and C4 will be completed during D2.

Data model, the D1 diagram – Component E4



[Boxes = Entity with attributes listed underneath; PK = primary key; FK = foreign key; numbers show cardinality; arrows show relationships.]

The model uses a relational structure because the entities have clear relationships. It allows primary and foreign keys to provide easier management through their connections.

Structural Decision	**Decision Rationale**
IDs = Integer primary keys or foreign keys	Simple identification that can be used throughout the entire project.
VideoAsset = Entity	Stores data for an uploaded demonstration video.
TrainingJob = Entity	Stores training involving the video with its status and progress.
ModelArtifact = Entity	Stores data for trained model output involving its own training.
PointCloudCache = Entity	Stores saved point-cloud data for a video.
VideoAsset -> TrainingJob = one-to-many	One video asset will have the option to be used for training multiple times.
VideoAsset -> PointCloudCache = one-to-many	One video asset will have the option to create multiple cache records, which will refer to only one video asset.
TrainingJob -> ModelArtifact = one-to-many	One training job may create multiple model artifacts, which will refer to only one training job.
Indexes = video_id, training_id, status	Easier to keep track of runs and check the status of training.
Data Model = Relational	Data is in a structured format with relationships clearly shown between entities.

E2:

Core algorithms -

Algorithm 1: Demonstration Video Sampling and Frame Validation

What	Video preprocessing: inspect the uploaded demonstration, sample frames at a configurable interval, normalize image dimensions, and preserve frame timestamps for downstream reconstruction.
I&O	Input: video_id (Integer >= 1); sample_fps (Float > 0, default 2); max_frames (Integer 2–300). Output: ordered RGB frames with timestamps (seconds), source dimensions, and frame count.
Complexity	O(F × P) time to decode and resize F sampled frames containing up to P pixels each; O(F × P) memory if retained together, or O(P) for streamed processing (excluding decoder buffers).
Why	Sampling reduces redundant adjacent frames while keeping enough viewpoints for reconstruction. Timestamped frames retain a connection to the source video.
Edge Cases	Reject missing/corrupt or zero-duration videos, unsupported codecs, insufficient distinct frames, invalid frame rates, and requests exceeding resource limits. Preserve aspect ratio; warn when motion blur or little camera movement may reduce reconstruction quality.

Algorithm 2: Point-Cloud Reconstruction with VGGT-Omega

What	Run the selected VGGT-Omega reconstruction model on sampled frames, estimate camera/scene geometry, and export a point cloud for later processing and caching.
I&O	Input: sampled RGB frames and reconstruction configuration (confidence threshold, frame selection). Output: point-cloud file (PLY/NPZ), estimated camera information when available, point count, and quality/status metadata.
Complexity	Model inference cost depends on input resolution, number of frames, model architecture, and GPU; treat it as model-dependent rather than claiming a fixed Big-O bound. Point filtering/export is O(N) for N predicted points.
Why	The team selected VGGT-Omega for fast feed-forward reconstruction across multiple views rather than building a geometry estimator from scratch.
Edge Cases	Low-texture scenes, reflective surfaces, occlusion, moving objects, poor viewpoint overlap, out-of-memory errors, and empty/low-

	confidence point clouds. Fail with a clear diagnostic or flag the output as low confidence.

Algorithm 3: Gaussian Splatting and Simulation-Ready Mesh Extraction

What	Use the reconstructed scene and camera estimates to fit a 3D Gaussian splatting (3DGS) representation; use the selected NVIDIA DNLR mesh-extraction approach to convert visual geometry into a mesh suitable for downstream simulation.
I&O	Input: reconstruction output, camera data, and 3DGS/mesh settings. Output: Gaussian-splat artifact, mesh artifact (e.g., OBJ/PLY), and metadata including vertex/face counts and completion status.
Complexity	3DGS optimization depends on iterations K, number of Gaussians G, and rendered pixels P; computational cost is implementation-dependent and grows with K and scene size. Mesh extraction and cleanup also depend on resolution and mesh size.
Why	3DGS supports realistic novel views, while the mesh provides explicit geometry needed for collision modeling. Reusing established reconstruction and extraction tools avoids implementing complex research models.
Edge Cases	Missing camera estimates, floating geometry, holes, disconnected components, non-manifold meshes, excessive polygon counts, scale ambiguity, and GPU exhaustion. Validate mesh integrity; report that visual reconstruction does not guarantee physically accurate collision geometry.

Build-versus-reuse decisions -

What	**B or R**	**Library/Service**	**License**	**Reason**
Video decoding / frame sampling	Reuse	FFmpeg / ffprobe	LGPL or GPL (build-dependent)	Reliable video inspection and frame extraction; avoid custom codec handling.

Pipeline orchestration and validation	Build	Python service code	Custom work	Connect video IDs, frame selection, reconstruction stages, error handling, and E4 artifact storage.
Point-cloud reconstruction	Reuse	VGGT-Omega (team-selected model)	Verify upstream model/weights license	Existing feed-forward geometry model; training a replacement is outside project scope.
Point-cloud filtering and export	Build + reuse	Python + Open3D (proposed)	MIT (Open3D); custom work	Implement project-specific confidence filtering while reusing point-cloud I/O and geometry operations.
Radiance-field reconstruction	Reuse	3D Gaussian Splatting implementation	Verify chosen repository license	Established 3DGS optimization/rendering rather than implementing the method from scratch.
Mesh extraction	Reuse	NVIDIA DNLR (team-selected approach)	Verify code and weights license	Convert splats into meshes using an existing research implementation.
E2 API integration	Build	FastAPI (proposed by E4)	MIT (FastAPI); custom work	Expose reconstruction requests and results with consistent /api/v1 validation and IDs.

API contract -

Proposed REST API: /api/v1. JSON requests/responses; binary outputs are stored through E4 and referenced by artifact IDs or storage keys. All IDs are positive integers. Reconstruction runs asynchronously because GPU processing can take time. Limits below are provisional.

Endpoint	Inputs	Outputs	Errors
POST /api/v1/videos/{video_id}/reconstructions	Path: video_id Int >= 1. JSON: sample_fps Float > 0 (default 2), max_frames Int 2–300 (default 100), generate_mesh Boolean (default true).	202: {reconstruction_id: Int, video_id: Int, status: "queued", progress_pct: 0}.	404 video not found; 409 duplicate active request (if deduplicated); 422 invalid parameters; 503 worker unavailable.
GET /api/v1/reconstructions/{reconstruction_id}	Path: reconstruction_id Int >= 1.	200: {reconstruction_id: Int, video_id: Int, status: "queued|running|completed|failed", progress_pct: Float 0–100, stage: String, error: String|null}.	400 invalid ID; 404 reconstruction not found; 500 metadata error.
GET /api/v1/reconstructions/{reconstruction_id}/artifacts	Path: reconstruction_id Int >= 1.	200: {reconstruction_id: Int, point_cloud: {cache_id: Int, storage_key: String}|null, gaussian_splat: {storage_key: String}|null, mesh: {storage_key: String}|null}.	404 reconstruction not found; 409 results not ready; 500 storage/metadata error.

PATCH /api/v1/reconstructions/{reconstruction_id}	Internal worker only. JSON: status enum queued/running/completed/failed; progress_pct Float 0–100; stage String.	200: {reconstruction_id: Int, status: String, progress_pct: Float}.	403 unauthorized worker; 404 not found; 409 invalid transition; 422 invalid values.

Integration: E2 reads uploaded VideoAsset metadata from E4; it registers generated point clouds using POST /api/v1/videos/{video_id}/point-clouds and stores other binary artifacts using E4's storage manager. The proposed reconstruction_id/status record and Gaussian-splat/mesh references require an E2-specific persistence extension, since the current E4 data model only defines VideoAsset, PointCloudCache, TrainingJob, and ModelArtifact. E3 consumes the validated mesh and/or reconstruction outputs for simulation setup.

E3:

Core algorithms -

Algorithm 1: Physics Model Training

What	Simulation Engine (MuJoCo) – Data from the simulations is used to train the physics model.
I & O	Inputs: 3D Mesh File, Outputs: Live simulation state (mjData), Live camera state (mjtCamOutBit), Summary(JSON), Status(String)
Complexity	Highly variable – depends on complexity of 3D mesh surfaces and scales with the number of objects and frequency of collision.

Why	MuJoCo was designed with robotics policy training in mind and is an incredibly advanced physics engine that is open source. Alternative option is Isaac Sim, however due to budget constraints we are not able to acquire the appropriate graphic cards needed to utilize Isaac Sim so it was dropped in favor of MuJoCo.
Edge Cases	It is possible for the input to be corrupted if video processing fails, so it is important for checks to be done to make sure the data is 'clean' before training the model, especially since training is a lengthy process. Empty input is automatically returned with an error to component E2. Duplicate inputs will need a check, either against current summaries stored in the backend, or checking status, to find other physics model(s) that have already been trained on the input.

Build-versus-reuse decisions -

What	B or R	Library/Serv	License	Reason
Physics Engine	Reuse	MuJoCo	Apache 2.0	Provides a physics engine that is able to read our produced mesh files easily. Creating our own physics engine is way out of scope.
Diffusion Model	Build	None	None	Diffusion model will be built instead of using a pre-existing one to allow us to tailor it more to the results from our physics engine and control the training process(this is maybe too general)

API contract –

Proposed REST API: /api/v1.

Endpoints	Inputs	Outputs	Errors
GET /api/v1/phys_model/{model_id}	model_id: path int >= 1. required	200: {summary: JSON, status: string}	400: invalid model id. 404: model not found
POST /api/v1/phys_model	muj_mjcf : MuJoCo XML. required	201: {model_id: int, summary: string}	500: Database error
GET /api/v1/diffusion_model/{model_id}	model_id image dataset training parameters	200: {summary: JSON, status: string}	400: invalid model id.
POST /api/v1/diffusion_model	model_id	201: {model_id: int, summary: string}	500: Training error

E4:

Core algorithms -

Algorithm 1: File Storage and Retrieval

What	File Storage and Retrieval. Accept a video or generated artifact as a stream, validate it, calculate its checksum, write it to storage, and save a linked metadata record. Retrieve files by stable record ID.
I & O	Input: file stream (binary); kind (String enum: video, point_cloud, model); associated ID (Integer >= 1, when required); content_type (String). Output: file ID (Integer), storage_key (String), size_bytes (Integer), sha256 (64-character hex String).

Complexity	O(S) time for reading/writing and SHA-256 hashing S bytes; O(B) memory using fixed 4 MiB chunks. Planning estimate: 20 videos x 200 MB = ~4 GB; at 100x, 2,000 videos = ~400 GB, excluding models and point clouds. File size, not database lookup, is the main scaling concern.
Why	Reuse the filesystem for binary objects instead of storing multi-gigabyte BLOBs in SQLite. Chunked transfer is simpler and uses less memory than loading each file in full.
Edge Cases	Reject empty/unsupported files. Deduplicate by checksum if policy requires; detect interrupted transfers and insufficient space; write to a temporary file and rename after validation; delete incomplete files and avoid dangling metadata rows on failure.

Algorithm 2: Training Record Tracking

What	Training Metadata and Status Management. Create a training record tied to a video, persist configuration and status, update progress, and associate model outputs and point-cloud references with their source records.
I & O	Input: video_id (Integer >= 1), parameters (JSON object), status (String enum), progress_pct (Float 0-100). Output: training_id (Integer), stored parameters (JSON object), status (String), progress_pct (Float), artifact IDs (Integer array).
Complexity	Indexed primary/foreign-key lookups and updates are typically O(log N) for N records. Illustrative workload: ~20 jobs initially and ~2,000 at 100x; the difference is modest compared with transferring large artifact files.
Why	SQLite transactions and foreign keys provide more reliable links among videos, training runs, caches, and artifacts than scattered JSON files; indexed access avoids scanning all job records.
Edge Cases	Invalid video IDs, empty parameters, concurrent updates, duplicate submissions, invalid status transitions, training failure, missing artifact files. Use transaction rollback when a metadata write fails.

Build-versus-reuse decisions -

What	B or R	Library/Serv	License	Reason

Database engine	Reuse	SQLite	Public domain	Mature relational store; no separate database service; appropriate for a small prototype.
Relational schema	Build	Custom SQL tables	Custom work	Defines VideoAsset, TrainingJob, ModelArtifact, and PointCloudCache relationships and indexes.
Database driver	Reuse	Python sqlite3	PSF License	Built into Python; supports SQL queries and transactions without extra infrastructure.
File storage manager	Build	Python + local filesystem	Custom work	Stores large video/model/point-cloud binaries separately from their database metadata.
Integrity / duplicate check	Reuse	Python hashlib (SHA-256)	PSF License	Stable checksum implementation for integrity checks and optional duplicate detection.
Video processing	Reuse	FFmpeg / ffprobe	LGPL or GPL (build-dependent)	Established tools to inspect video attributes and optionally create compressed MP4 copies; retain originals when accuracy matters.
Backend API framework	Reuse	FastAPI (proposed)	MIT	Typed validation and documented HTTP endpoints; fits a Python-oriented ML stack.
E4 API and validation rules	Build	Custom Python handlers	Custom work	Implements SkillForge-specific file limits, job state transitions, and metadata-to-file links.

Note: Library maturity, license details, and performance should be checked against the versions the team selects, especially FFmpeg builds.

API contract -

Proposed REST API: /api/v1; JSON for metadata and multipart/form-data for binary uploads. All IDs are positive integers. File-size limits and training parameter ranges are provisional.

Endpoint	**Inputs**	**Outputs**	**Errors**

POST /api/v1/videos	file: Binary, required MP4/MOV, 1 byte-2 GiB.	201: {video_id: Int, storage_key: String, size_bytes: Int (bytes), sha256: String}.	400 unsupported/empty file; 409 checksum duplicate (if dedupe enabled); 413 over limit; 500 storage error.
GET /api/v1/videos/{video_id}	video_id: path Int >=1, required.	200: {video_id: Int, storage_key: String, size_bytes: Int (bytes), duration_s: Float|null (seconds)}.	400 invalid ID; 404 video not found; 500 metadata error.
POST /api/v1/training-jobs	video_id: Int >=1, required; parameters: Object required with epochs Int 1-1000, learning_rate Float >0 and <=1, seed optional Int 0-2147483647.	201: {training_id: Int, video_id: Int, status: "queued", progress_pct: Float (0)}.	404 source video missing; 422 invalid parameters; 500 database error.
GET /api/v1/training-jobs/{training_id}	training_id: path Int >=1, required.	200: {training_id: Int, video_id: Int, parameters: Object, status: String, progress_p	400 invalid ID; 404 training job missing; 500 database error.

		ct: Float (0-100)}.	
PATCH /api/v1/training-jobs/{training_id}	training_id: path Int >=1; status: required String enum queued/running/completed/failed; progress_pct: Float 0-100. Restricted to internal training worker.	200: {training_id: Int, status: String, progress_pct: Float (0-100)}.	404 job missing; 409 invalid transition; 422 invalid progress/status; 500 error.
POST /api/v1/training-jobs/{training_id}/artifacts	training_id: path Int >=1; file: required nonempty binary; model_type: required String length 1-64.	201: {model_id: Int, training_id: Int, storage_key: String, size_bytes: Int (bytes)}.	404 job missing; 413 size limit exceeded; 422 missing model_type; 500 storage error.
POST /api/v1/videos/{video_id}/point-clouds	video_id: path Int >=1; file: required nonempty PLY/NPZ binary.	201: {cache_id: Int, video_id: Int, storage_key: String, size_bytes: Int (bytes)}.	400 unsupported format; 404 video missing; 413 size limit exceeded; 500 storage error.

6. Technology choices with justification –

3D Reconstruction Methodology – SkillForge will use a dual-component 3D reconstruction approach which will generate both a point cloud and a radiance field representation from input images. Generating both data representations, while more computationally expensive, allows for using the advantages from both. Point clouds can

be used as priors for radiance field generation as well as provide a representation best for extracting measurements of segmented 3D objects in the images. Radiance fields are best used for photo-realistic image generation at novel viewing angles which is best for ensuring visual accuracy for simulated robotics training environments. No impact on hosting.

Point-Cloud Generation – Point clouds will be generated using the VGGT-Omega foundational reconstruction model. This model provides a fast and accurate way to generate point clouds from one or more viewing angles of an image. This is chosen over alternatives due to the speed of generation (a single model forward pass vs complex statistical optimization algorithms) and ability to perform on a wide variety of images out-of-the-box. VGGT-Omega also uses only modest VRAM which means it does not require particularly expensive GPU access.

Radiance Field Generation – The radiance field representation and generation method SkillForge will use is 3D gaussian splatting (3DGS). This representation is chosen over an alternative such as Neural Radiance Fields due to the speed of reconstruction production provided by 3DGS. Additionally, the team is familiar with open-source libraries and tools which process 3DGS data structures for integrating into physics engines and development into 3D meshes. No impact on hosting.

3D Mesh Extraction – This is the task of taking the visual-fidelity focused representation of the 3DGS and producing a 3D-mesh which can be ingested into a physics engine and be used for collision simulation. To do this, we will use the DNLR foundation model from NVIDIA. This is an open-source model built to predict density maps from gaussian splats with the explicit goal of then extracting high-quality 3D meshes. This is chosen due to impressive results seen in NVIDIA demos and the ease of using the model through existing NVIDIA python libraries. No impact on hosting. No impact on cost. No impact on licensing. No impact on community support. No impact on performance. No impact on team skill fit.

Physics Engine – When choosing which physics engine to use for robotics simulation, we found the 2 most common choices are Isaac-Sim and MuJoCo. We have chosen to utilize

MuJoCo for the project due to having much faster rendering speeds. While Isaac-Sim is generally regarded for producing higher-quality visual rendering, it is also built primarily for industry-grade NVIDIA graphics cards which go beyond the budget for this project. No impact on hosting. No impact on cost. No impact on licensing. No impact on community support. No impact on performance. No impact on team skill fit.

Backend Language – Our project will use Python for the backend language. This is done due to all project members having experience working with Python, and because the community support and available tooling for ML-based workflows are predominately built in Python. Python and most associated tools are also free to use. While known as a slow language, the majority of backend time complexity will be held in highly optimized libraries with implementations in C or CUDA, making Python a relatively performant option. No impact on hosting.

Backend Web Framework – Our project will use FastAPI + Pydantic typing for the backend api construction. FastAPI is built for our language of Python and has strong community support. As the name implies, the framework was built to prioritize speed compared to competitors like Django and Flask. This benefits considerations like speed, cost, and hosting cost. Additionally, FastAPI's integration with Pydantic typing allows for auto-generation of API documentation. Several team members have prior experience with the framework. No impact on hosting.

Database – The group will use SQLite as the primary backend for the project. SQLite provides a lightweight database which integrates cleanly with the existing Python-based backend. The main benefit of SQLite is that it is free and easy to setup, and that several team members have prior experience with the database. One downside is that SQLite may not scale as well as more robust purpose-built databases. No impact on hosting.

Frontend Stack – For the frontend stack, SkillForge will utilize Typescript + React. This provides a widely used and freely available stack which the team has prior experience using for frontend projects. No impact on hosting. No impact on performance.